Week 3 Technical Report

TensorFlow / Keras: porting a PyTorch model, and a CPU-bound CIFAR-10 project

1. Introduction

This week covers two pieces of work and one report. Task 3.1 rebuilds the Week-2 Fashion-MNIST CNN in TensorFlow/Keras and compares it against the PyTorch original, reaching **91.40%** test accuracy against **90.67%**. Task 3.2 is a self-contained CIFAR-10 image classification project, reaching **76.84%** test accuracy. The connecting thread is honesty about what the numbers do and do not support: both tasks produced a result that could have been dressed up and was not — and in 3.2 the measurement overturned a prediction I had already written down.

2. Setup

Item	Task 3.1	Task 3.2
Dataset	Fashion-MNIST	CIFAR-10
Split	54,000 / 6,000 per class	45,000 / 5,000 stratified
Test set	10,000, opened once	10,000, opened once
Framework	TF 2.21.0 / Keras 3.15.1	TF 2.21.0 / Keras 3.15.1
Parameters	421,738	120,810
Optimizer	Adam 0.001	SGD 0.05, Nesterov, cosine
Schedule	8 epochs, no schedule	20 epochs, label smoothing 0.1
Hardware	CPU only	CPU only

TensorFlow has had no GPU path on native Windows since 2.11, so both tasks are CPU-bound. That single constraint dictated the model size in 3.2 and is the honest reason its accuracy sits 16 points below a standard baseline.

3. Task 3.1 — porting the Week-2 CNN

The objective was a like-for-like port, so the acceptance criterion was the parameter count, not the accuracy. Matching **421,738** against PyTorch's **421,738** required disabling the Keras *Conv2D* bias, which *BatchNormalization* already makes redundant.

Run	Test accuracy	Note
Week 2, PyTorch	90.67%	reference
Week 3, TensorFlow	91.40%	+0.73 points
Same recipe, new seed	—	0.27-point best-val gap

The third row is the important one. Rerunning the identical recipe with a fresh seed moved best-validation accuracy by 0.27 points with the framework held completely fixed. The test-set gap is +0.73 points, roughly 2.7× that spread — suggestive, but two runs cannot separate "better framework" from "luckier seed". The reportable conclusion is that the two frameworks are indistinguishable at this scale, and that differences below about a point here carry no information.

3.1 A parameter-counting discrepancy worth reporting

The first comparison reported 421,930 Keras parameters against PyTorch's 421,738 — a gap of 192 on what is provably the same architecture. *model.count_params()* sums *layer.weights*, and *BatchNormalization* carries four tensors per channel: learnable *gamma* and *beta* plus non-learnable *moving_mean* and *moving_variance*. PyTorch's *model.parameters()* yields trainable tensors only, so the two frameworks disagree by exactly 2×C_bn = 2×(32+64) = 192. Comparing *trainable_weights* against *parameters()* resolves it. This is the same buffer-versus-parameter distinction that had caused an undercount in Week 2, so it is documented rather than quietly patched.

4. Task 3.2 — CIFAR-10 project

A four-convolution CNN with BatchNorm, dropout and global average pooling, **120,810** parameters, trained for 20 epochs with SGD and Nesterov momentum on a cosine schedule. It was built with an image augmentation pipeline as part of the recipe, and the first result was 65.91% test accuracy. Measuring whether that pipeline earned its cost changed the answer: without it the same model reaches **76.84%**. The final model for this task is the no-augmentation variant, and the augmented run is retained as the experiment that justifies that choice.

Training runs from a resumable script that checkpoints every epoch — necessary at 210 seconds per epoch — with the learning rate expressed as a function of the absolute epoch index, so a resumed run is identical to an uninterrupted one.

4.1 Per-class results, and why recall alone misleads

Class	Support	Recall	Precision
dog	1000	60.60%	75.09%
cat	1000	64.90%	59.22%
horse	1000	65.30%	94.91%
bird	1000	65.60%	67.01%
deer	1000	74.30%	71.51%
airplane	1000	80.30%	79.50%
automobile	1000	86.20%	94.10%
truck	1000	88.30%	87.95%
ship	1000	91.20%	83.98%
frog	1000	91.70%	66.69%

In the augmented model, *frog* looked like the best class: 96.10% recall on 35.80% precision, predicted 2,684 times for 1,000 actual frogs. It was absorbing every animal it could not identify, and ranking classes by recall would have put its worst over-predicter first. The final model still over-predicts frog (1,375 times, 66.69% precision) but far less severely, and its weakest classes are now *dog* (60.60%) and *cat* (64.90% recall, 59.22% precision — the worst precision in the model) confused with each other. The confusion structure changes shape entirely between two runs of the same architecture, which is a good reason to read per-class precision rather than file a single accuracy figure and move on.

4.2 Does the augmentation earn its cost?

A three-epoch ablation put augmentation *behind* by 7.79 points. I recorded that as inconclusive on the grounds that a short ablation cannot see where a regulariser pays off, and wrote that the augmented run would overtake later. That caution was reasonable; the prediction built on it was wrong. So the control was rerun for the full 20 epochs with the pipeline switched off and everything else identical.

Setting	Best val	Best epoch	Test
Augmented	67.73%	18	65.91%
No augmentation	78.89%	17	76.84%
Difference	-11.16 pts	—	-10.93 pts

Augmentation costs **10.93 points** of test accuracy. The plain variant is ahead at every epoch and the gap widens with training, so this is not an early deficit that reverses — the augmented run never catches up. The likely reason is capacity rather than a broken pipeline: the ops are standard (flip, translation by 4/32 of the image, zoom, brightness and contrast at ±10-15%), applied per batch to the training split only, and both arms share every code path except the flag. At 120,810 parameters the network is already underfitting, and a regulariser applied to an under-parameterised model pushes it further from a fit it was struggling to reach. Augmentation is not the fix here; depth or width is.

5. Key takeaways

- **Match the architecture before comparing results.** A parameter-count mismatch makes an A/B comparison meaningless while still looking entirely reasonable. The 192-parameter gap was framework bookkeeping, and finding it was worth more than the accuracy delta.
- **Measure the noise floor before claiming a winner.** A same-recipe repeat costing one extra run turned "TensorFlow is better" into "indistinguishable at this scale".
- **Scepticism cuts both ways.** The 3-epoch augmentation ablation said plain was ahead by 7.8 points, and I discounted it as too short to conclude. The full control came out -10.9 points — the same direction, twice the size. Discounting a result because it is inconvenient is its own failure mode; the fix is to measure it properly, not to assume the convenient answer.

• **Watch the metric that flatters you.** Frog had the best recall in the CIFAR-10 model and one of the worst precisions, predicted 2,684 times for 1,000 actual frogs. Per-class recall without precision inverts that story, and in fashion-MNIST Shirt and T-shirt/top absorb each other the same way.

6. Limitations

The CIFAR-10 model is small and it shows. 76.84% against the ~93% a ResNet-18 (~11M parameters) commonly reports. The architecture was sized to finish on CPU; it is the binding constraint on accuracy, not the recipe.
One seed per arm. The noise floor was measured at 0.27 points on Fashion-MNIST; CIFAR-10 is noisier still. The ~11-point augmentation gap is far outside that, so its sign is safe, but its magnitude is a single sample and the augmented arm's exact accuracy deserves a second run before being quoted to three decimal places.
The augmentation result is specific to this model, not to CIFAR-10. It shows the pipeline is misjudged for a 120k-parameter network that is already underfitting. It is not evidence against augmentation in general, which is expected to help once the network is large enough to be underfitting-free.
Timing is contaminated. A few epochs lost most of their CPU to other work on a shared machine and ran several times slow, which inflates the mean epoch time; the median (210 s) is the honest figure.
The test sets were opened once each, after every decision had been made against validation, so neither test number is fitted — but neither is a cross-validated estimate either. Opening the test set for the control's evaluation was a second look at the same 10,000 images, which is a mild caveat on the comparison being independent.

Generated by *build_report.py* directly from the task artifacts. No figure in this report was typed by hand; regenerate with *python build_report.py*.